

PLANT DISEASE DETECTION & ANALYSIS SYSTEM

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS

2023 - 26



Done By,

Gautham Nair (Reg. No: 230021080526)

Under the guidance of

Ms. Dona Maria Mani

**RAJAGIRI COLLEGE OF MANAGEMENT AND
APPLIED SCIENCES**

(Affiliated to Mahatma Gandhi University, Kottayam)

Rajagiri Valley P.O, KERALA- 682039

RAJAGIRI COLLEGE OF MANAGEMENT AND APPLIED SCIENCES

(Affiliated to Mahatma Gandhi University, Kottayam)

Rajagiri Valley P.O, KERALA- 682039



CERTIFICATE

*This is to certify that the mini project work titled “Plant Disease Detection & Analysis System” submitted to Mahatma Gandhi University in partial fulfillment of the requirements for the award of the Degree of Bachelor Of Computer Applications is a record of the original work done by **Gautham Nair** under my supervision and guidance and that this project work has not formed the basis for the award of any Degree/ Diploma/ Fellowship or similar title to any candidate of this or any other University*

Faculty In-Charge

Project Coordinator

Head of the Department

Submitted for viva-voce held on/...../.....

Internal Examiner

External Examiner

RAJAGIRI COLLEGE OF MANAGEMENT AND APPLIED SCIENCES

(Affiliated to Mahatma Gandhi University, Kottayam)

Rajagiri Valley P.O, KERALA- 682039



DECLARATION

I **GAUTHAM NAIR** hereby declare that project report entitled “**PLANT DISEASE DETECTION & ANALYSIS SYSTEM**” submitted in partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Applications is a record of original research work done by us under the supervision & guidance of **Ms. Dona Maria Mani** and the dissertation has not formed the basis for the award of any Degree/Diploma/ Associate ship / Fellowship or similar title to any candidate of this or any other University.

Gautham Nair

Place: Kakkanad

Date:...../...../.....

ACKNOWLEDGEMENT

I consider it as a privilege to express our sincere gratitude and respect to all those who guided and inspired us in the successful completion of this project work.

We convey our reverential salutation to **Almighty God**, for enabling us to take up and complete the project successfully.

I would like to express our sincere thanks to **Rev. Fr. Ajeesh Puthussery CMI**, Director and **Prof. Dr. Laly Mathew**, Principal, Rajagiri College of Management and Applied Sciences for providing the necessary infrastructure and support for the completion of this project work.

We would like to express our sincere thanks to **Ms. Nileena Sasidharan**, HOD, PG Department of Computer Science, Rajagiri College of Management and Applied Sciences for her valuable advice and support which have helped us greatly in the accomplishment of the project.

I would like to express our sincere thanks to the Project Coordinator **Mr. Joby Jacob**, Assistant Professor, PG Department of Computer Science, Rajagiri College of Management and Applied Sciences for his consistent guidance and inspiration throughout the period for the completion of our project.

I would like to express my sincere thanks to the Project Guide Ms Dona Maria Mani, Assistant Professor, PG Department of Computer Science, Rajagiri College of Management and Applied Sciences for her consistent guidance and inspiration throughout the period for the completion of my project.

I would like to thank all the teaching and non-teaching staff of Rajagiri College of Management and Applied Sciences, for their valuable guidance and suggestions rendered during the project.

Finally, I thank our parents and all our friends for their help, encouragement and moral support given to us during the course of this work.

Gautham Nair

TABLE OF CONTENTS

1. INTRODUCTION	1
1.1 About The Project.....	1
1.2 About The Organization	2
2. REQUIREMENT ANALYSIS AND SPECIFICATION	3
2.1 System Study.....	3
2.1.1 Existing System.....	4
2.1.2 Proposed System	4
2.1.3 Feasibility Study.....	5
2.2 User Characteristics	7
2.3 System Specification.....	8
2.3.1 Hardware Specification	8
2.3.2 Software Specification	8
2.3.3 About The Software Tools And Platform.....	9
3. SYSTEM MODELING	11
3.1 Modules And Description	11
3.2 Data Flow Diagram.....	14
3.2 Entity Relationship Diagram.....	21
4. SYSTEM DESIGN	23
4.1 Input Design.....	23
4.2 Output Design	28
4.3 Database Design.....	31

5. TESTING	39
5.1 Introduction.....	39
5.2 Test Cases	42
6. IMPLEMENTATION	45
6.1 Introduction.....	45
6.2 Installation Procedure.....	46
6.3 Implementation Plan.....	48
7. CONCLUSION	49
7.1 Future Enhancements	49
BIBLIOGRAPHY	50
APPENDICES	
APPENDIX A	52
Sample source code/pseudo code Login Page Code	52
APPENDIX B	55
Acronyms	

INTRODUCTION

1.1 ABOUT THE PROJECT

The project "**Plant Disease Detection and Analysis System**" is an online platform developed for farmers, agricultural students, and researchers to identify plant diseases instantly by uploading leaf images. The aim is to provide accurate, automated disease detection for major crops like Cotton, Rice, Wheat, Maize, and Sugarcane, making professional-grade diagnosis accessible through a dedicated web application.

In today's agricultural landscape, timely disease identification is crucial for preventing crop loss and ensuring food security. This platform combines advanced deep learning technology with a user-friendly interface, enabling users to get instant predictions with confidence scores and treatment recommendations. By digitizing the identification process, the system offers a seamless experience where users can upload images, view detailed predictions, explore Top-K alternative diagnoses, and receive comprehensive treatment advice.

The system includes an admin panel for managing models and configuration, ensuring quality control and optimal performance. The project emphasizes professional design, easy usability, and secure functionality. With features like predictions, confidence scoring, Top-K alternatives, treatment recommendations, and prediction history tracking, the system bridges the gap between AI technology and practical agricultural needs, making expert-level disease detection available to a global audience.

1.2 ABOUT THE ORGANIZATION

Rajagiri College of Management and Applied Sciences (RCMAS) is a premier institution of higher education located in Kakkanad, Kochi, Kerala. Established in 2005, the college operates under the esteemed Rajagiri Vidyapeetham and is managed by the Carmelites of Mary Immaculate (CMI) congregation, known for their dedication to quality education and social commitment. The college is affiliated with Mahatma Gandhi University, Kottayam, and offers a wide range of undergraduate and postgraduate programs in fields such as Commerce, Management, Computer Applications, English, and Graphic Design. RCMAS is committed to academic excellence, character formation, and the holistic development of students, guided by its motto "Learn, Serve, Excel." The campus is situated in the picturesque Rajagiri Valley, an eco-friendly environment that supports both academic and personal growth. RCMAS fosters a vibrant learning community through well-qualified faculty, modern infrastructure, and numerous clubs and student organizations that promote creativity, leadership, and service. With a strong emphasis on innovation and ethical values, the college has earned a reputation for producing socially responsible professionals equipped to meet the demands of a dynamic global environment. Over the years, the institution has achieved significant academic success, including multiple university ranks and recognitions, reinforcing its position as a leading center for higher education in Kerala..

REQUIREMENT ANALYSIS AND SPECIFICATION

2.1 SYSTEM STUDY

The Plant Disease Detection and Analysis System is designed to streamline and automate the full workflow of plant health diagnosis, from user registration and image upload to AI-based prediction, confidence scoring, treatment guidance, and downloadable reports. It supports role-based operations through member and admin modules, where users can track prediction history, manage profiles, and upgrade plans, while administrators oversee users, payments, model performance, logs, notifications, and database status.

The proposed system provides a centralized, web-based platform that reduces delays and inaccuracy from manual plant disease identification and fragmented communication channels. With responsive interfaces, secure authentication, subscription-based feature access, and integrated reporting, the system improves decision-making for farmers, researchers, and agricultural stakeholders while maintaining operational control and scalability for administrators.

The main components of building this software are:

- Requirement analysis and AI workflow/system design
- Implementation using secure, scalable web and ML technologies
- Verification, testing, and deployment of prediction and admin modules
- Continuous maintenance, model updates, monitoring, and documentation

2.1.1 Existing System

These systems lack integration between disease identification, treatment recommendations, and prediction history. Manual methods lead to misdiagnosis, delayed treatment, crop loss, and trust issues. Farmers cannot verify disease accuracy, and there is no mechanism for confidence scoring, alternative diagnoses, or centralized disease database. There's also no automated prediction system — making it frustrating for users to get accurate disease identification. Traditional methods of plant disease identification rely on disorganized systems such as:

- Manual field inspections by agricultural officers
- Physical consultations with experts
- Online image searches and guess-based diagnosis

Disadvantages of Existing System

i) Time-consuming manual data entry and attendance tracking. ii) No centralized access to session schedules, coach assignments, or student details. iii) Higher chance of data loss, errors, or duplicate entries in paper registers. iv) Manual and unreliable communication processes for schedule changes. v) Difficulty generating accurate student attendance or coach activity reports.

2.1.2 Proposed System

The proposed Plant Disease Detection and Analysis System is a web-based platform that integrates all key functions into a seamless user interface. Users can sign up, upload leaf images, get instant disease predictions with confidence scores, view Top-K alternative diagnoses, and access treatment recommendations. Additionally, administrators have full control over model management, system configuration, and database monitoring.

The system supports secure logins, encrypted passwords, personalized profiles, and clean navigation. Key features include instant AI-powered disease detection using custom-trained MobileNetV2 model, confidence scoring and Top-K predictions for reliability, comprehensive treatment recommendations with symptoms and descriptions, prediction history tracking in SQLite database, admin dashboard with model and system management, search and filtering capabilities for prediction history, and secure image upload with preprocessing and validation.

The advantages of the proposed system include reducing operational friction, improving disease identification accuracy and traceability, centralizing disease information and prediction tracking, providing scalability for future growth, and encouraging data-driven agricultural decisions with AI oversight.

Advantages of Proposed System

i) Centralized and digital management of the organizational hierarchy, users, and schedules. ii) Automated attendance tracking and real-time report generation. iii) Easy access to personal schedules for coaches and students. iv) Efficient communication through automated notification alerts. v) Minimizes paperwork and reduces data entry errors.

2.1.3 Feasibility Study

The feasibility study evaluates whether the Plant Disease Detection and Analysis System project is practical and viable to develop and implement. It covers three key aspects: technical, economic, and operational feasibility. Technical feasibility assesses whether the required technology, such as Python (Flask), TensorFlow/Keras, and SQLite, is available and suitable for the project, and if the system can run efficiently on standard hardware. Economic feasibility examines the costs involved in development, deployment, and maintenance, confirming that using open-source tools and minimal hosting expenses make the project affordable and cost-effective.

Operational feasibility looks at how easily the system can be used by farmers, students, researchers, and administrators, ensuring the user interface is simple, responsive, and requires minimal training. Together, these assessments confirm that the project is achievable, sustainable, and beneficial for all stakeholders.

2.1.3.1 Technical Feasibility

The project uses Python (Flask) for both backend and frontend (templates), TensorFlow/Keras for AI model processing, and SQLite for data storage. It runs on standard configurations with minimal hardware requirements. Front-end components are built using Python Flask templates with HTML, CSS, and JavaScript to provide interactivity and responsiveness. The backend structure supports modularity and scalability, with a lightweight MobileNetV2 model (~17MB) ensuring fast inference on standard hardware.

2.1.3.2 Economic Feasibility

The system is highly economical because all tools and platforms used are free and open-source. Hosting costs are minimal for a small to mid-sized platform. The operational savings gained through automation outweigh the initial development efforts. Additionally, the system reduces the need for manual inspections and agricultural officer visits, which lowers long-term operational costs and prevents crop loss through timely disease detection.

2.1.3.3 Operational Feasibility

The system is designed for ease of use. Users interact with the platform through clean forms and simple interfaces, while the admin panel features structured controls and dashboards. Minimal training is required to use the system effectively. The user interface is responsive, intuitive, and optimized for various devices such as desktops, tablets, and smartphone.

2.2 USER CHARACTERISTICS

The Plant Disease Detection and Analysis System is designed to provide users with an easy and efficient way to perform their tasks while minimizing complexity. This system ensures that users can rely on it for smooth operations and desired outcomes. There are two main users in this system:

- Administrator
- Customer

2.2.1 Administrator

The Administrator is the main controller of the system. They oversee all activities including model management, database configuration, and system monitoring. The Administrator has full control over the system's operations and ensures everything runs smoothly. They can manage model files, configure system parameters, access all prediction data, and retrain models if needed.

2.2.2 Customer

The Customer (Farmer/Student/Researcher) is the primary beneficiary of the system. They can upload leaf images, view disease predictions with confidence scores, access Top-K alternative diagnoses, and read treatment recommendations. Their role ensures that the system serves its main purpose of providing instant, accurate disease detection to those who need it most.

\

2.3 SYSTEM SPECIFICATION

2.3.1 Hardware Specification

The selection of hardware is very important in the existence and proper working of any software. When selecting the hardware, the size and capacity requirements are also important. Below is some of the hardware that is required by the system.

Processor	Intel Core i5-3220 (3.3 GHz) or above
RAM	4 GB
Storage	512 GB and above
Other Peripherals	Standard Keyboard and Mouse

2.3.2 Software Specification

Operating System	Windows 7/8/8.1/10/11 or Linux-based servers
Front End	Python (Flask Templates) with HTML, CSS,
Back End	Python (Flask) with TensorFlow/Keras, SQLite

2.3.3 About the Software Tools and Platform

It is very important to select the appropriate software so that the software works properly.

Below is the software's that are required to make a new system:

Python (Flask Framework)

TensorFlow/Keras

SQLite

FRONT END SPECIFICATION: Python (Flask) and TensorFlow/Keras

Python (Flask Framework):

Python with Flask is a lightweight and efficient web framework used to build server-side logic and handle routing. Flask supports session handling, file uploads, and interaction with databases, making it suitable for developing scalable web applications. It can create RESTful APIs, handle image processing, and interface with AI models. Flask follows a minimalistic design, allowing rapid application development while keeping the system easy to maintain.

TensorFlow/Keras

TensorFlow/Keras is an integrated deep learning and machine learning solution that delivers increased accuracy, scalability and availability to AI applications, while making them easier to build, deploy and manage. It enables reliable management of model files and confident execution of complex image classification tasks. TensorFlow/Keras provides high performance with the highest levels of prediction accuracy, proven scalability for handling large amounts of image data, secure model file protection, automated diagnostics and checkpointing, and comprehensive analysis tools for enhanced decision making

BACK END SPECIFICATION: SQLite

SQLite is a lightweight, file-based database management system that delivers increased reliability, portability and availability to data storage applications, while making them easier to build, deploy and manage. It enables reliable management of prediction history and confident execution of complex data storage requirements. SQLite provides high availability with data protection against errors and data loss, proven scalability for handling large amounts of prediction data, secure data protection against unauthorized access, automated diagnostics and indexing, and comprehensive analysis tools for enhanced decision making.

SYSTEM MODELING

3.1 MODULES AND DESCRIPTION

Plant Disease Detection and Analysis System provides users with a platform to identify plant diseases online. This system is developed to manage the overall operations of an AI-powered disease detection platform, specifically focused on accurate identification of diseases and pests affecting Cotton, Rice, Wheat, Maize, and Sugarcane crops. It allows users to upload images, get predictions, and access treatment information. There are 7 modules in this project.

- 1. : Customer Registration**
- 2. : Subscription Management**
- 3. : Payment Management**
- 4. : Disease Information Management**
- 5. : Disease Prediction Management**
- 6. : Disease Analysis Management**
- 7. : Notification Management**

1. Customer Registration

The Customer Registration Management module is designed to handle new customer registrations and maintain customer profiles. Customers can securely create accounts using their email and password, and returning customers can log in to access disease detection services, prediction history, and subscription management.

2. Subscription Management

The Subscription Management module controls all user subscription activities. It ensures that subscription statuses are updated automatically after successful payments and allows users to manage or modify their current plans.

3. Payment Management

The Payment Management module handles all monetary transactions between customers and the server. It processes Pro subscription payments, generates receipts, and stores complete payment histories for auditing and user reference.

4. Disease Information Management

This module manages comprehensive disease information and treatment recommendations for all detected crop diseases. Users can access detailed disease descriptions, symptoms, treatment recommendations, Top-K alternative diagnoses, and view supported plants for both Basic and Pro plans.

5. Disease Prediction Management

This module handles the complete disease detection lifecycle—from image upload to prediction results. Customers can upload crop leaf images, and the system processes them through machine

learning models to return disease predictions with confidence scores. Pro users receive multi- model ensemble results, while free users get single model predictions.

6. Disease Analysis Management

The Disease Analysis Management module provides users with comprehensive analysis of their disease detection activities. Users can view their prediction history, access detailed

statistics, analyze trends, and manage their account settings with usage information and account type details.

7. Notification Management

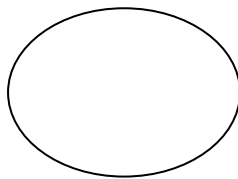
This module handles all automated and manual alerts within the CropGuard system. It automatically sends notifications about premium plan expiry reminders, usage limit resets, subscription updates, and allows administrators to create and send announcements to all users or specific groups.

3.2 DATA FLOW DIAGRAM

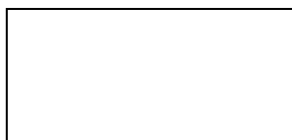
A data flow diagram is graphical tool used to describe and analysis movement of data through a system. These are central tool and the basis from which the other components are developed. The transformation of data from input to output, trough processed, may be described logically and independently of physical components associated with the system. These are known as the logical data flow movement of data between people, departments and workstations. A full description of a system actually consists of a set of data flow diagrams.

A DFD is also known as a “bubble chart” has the purpose of clarifying system requirements and identifying major transformations that will become programs in system design. So, it is the starting point of the design to the lowest level of detail. A DFD consists of a series of bubbles joined by data flows in the system.

In the DFD, there are four symbols:



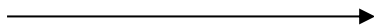
Process that transforms dataflow



Source or Destination of data



Data Store



Information or Dataline

Rules for drawing data flow diagrams:

Rule 1: Define the context of the diagram by identifying all input and output data flows.

Rule 2: Choose a starting point for drawing the DFD.

Rule 3: Provide meaningful labels for all data flow lines.

Rule 4: Label processes with action verbs that relate to input and output data flows.

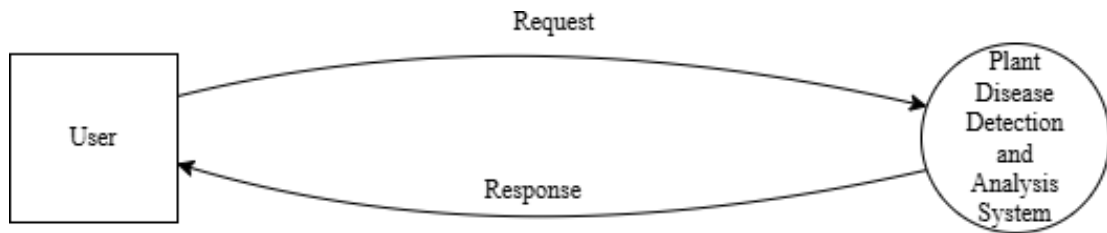
Rule 5: Exclude insignificant functions usually handled during programming.

Rule 6: Avoid including control or flow of control information.

Rule 7: Avoid overcrowding the DFD with excessive information.

Rule 8: Be open to starting the diagram over if necessary.

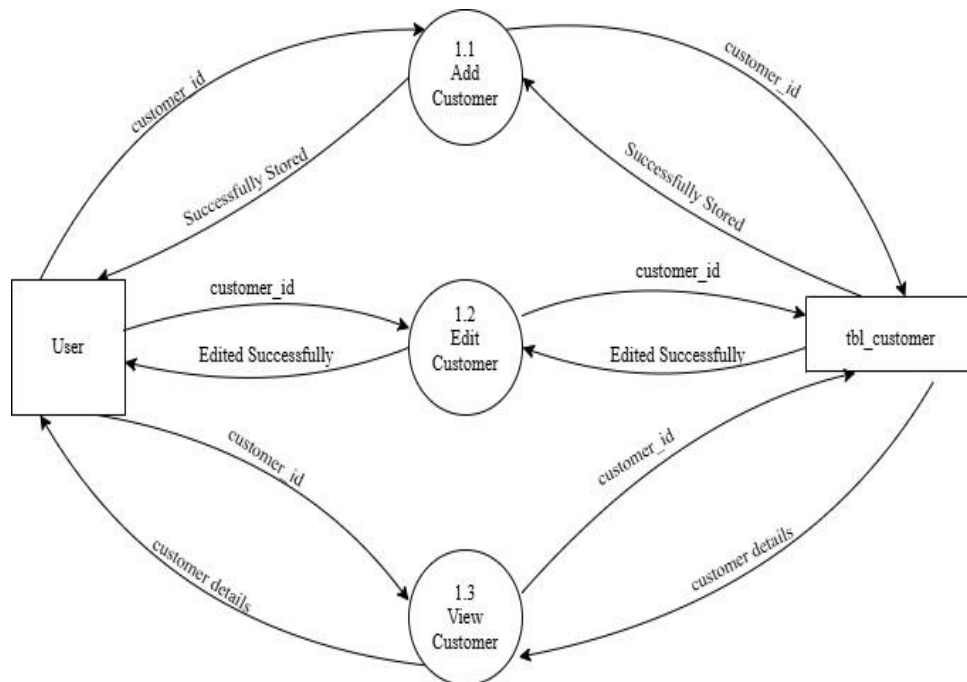
Level 0 DFD- Plant Disease Detection and Analysis System



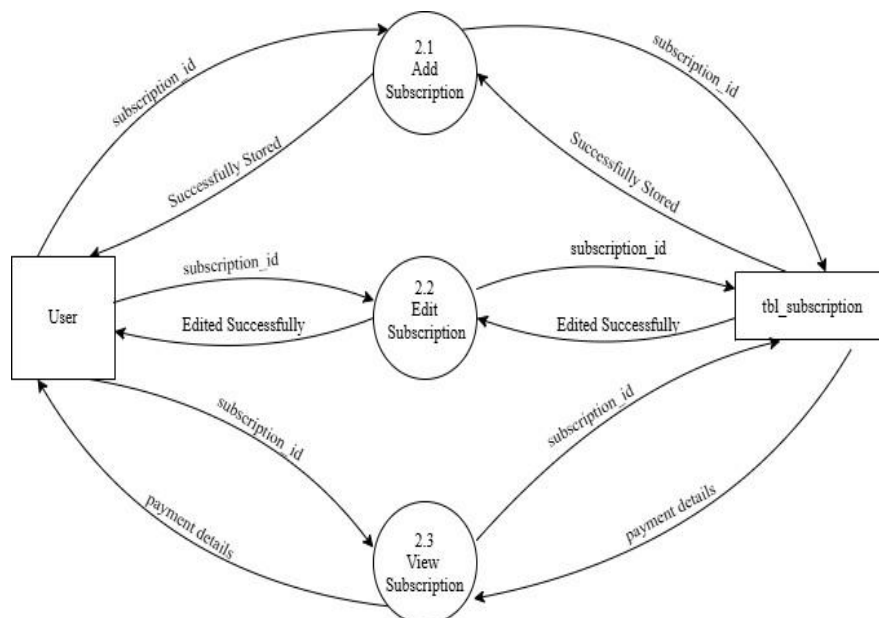
Level 1 DFD- Plant Disease Detection and Analysis System



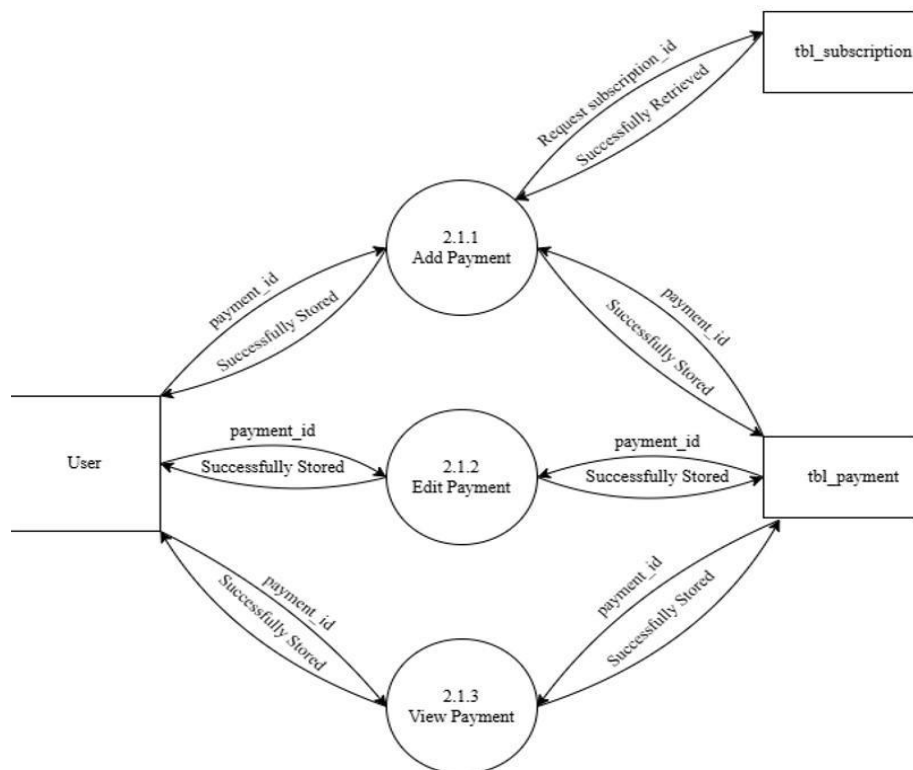
Level 2 DFD: Customer Management



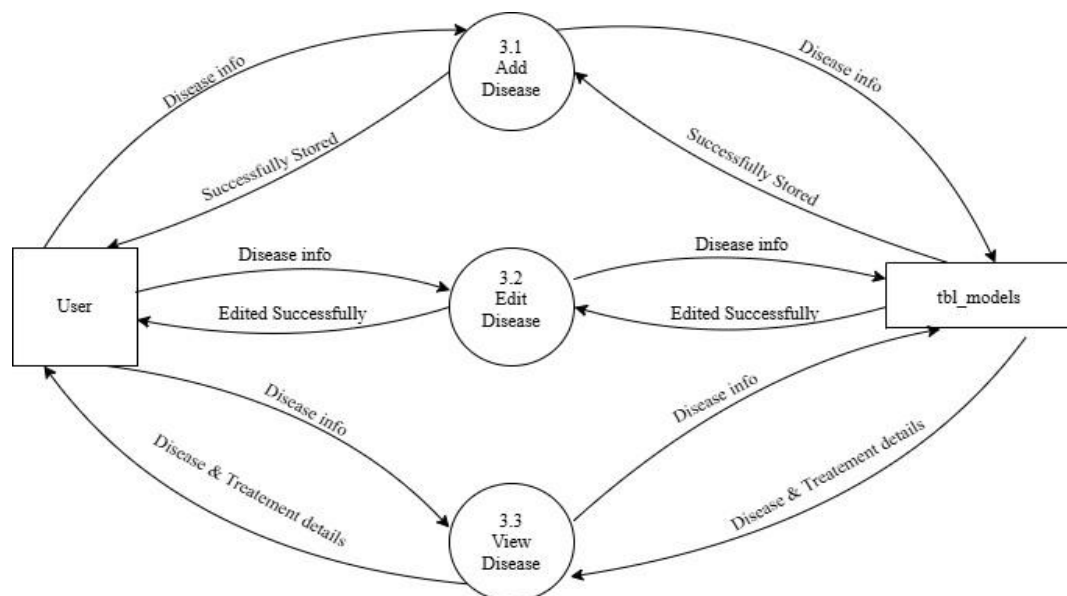
Level 2 DFD: Subscription Management



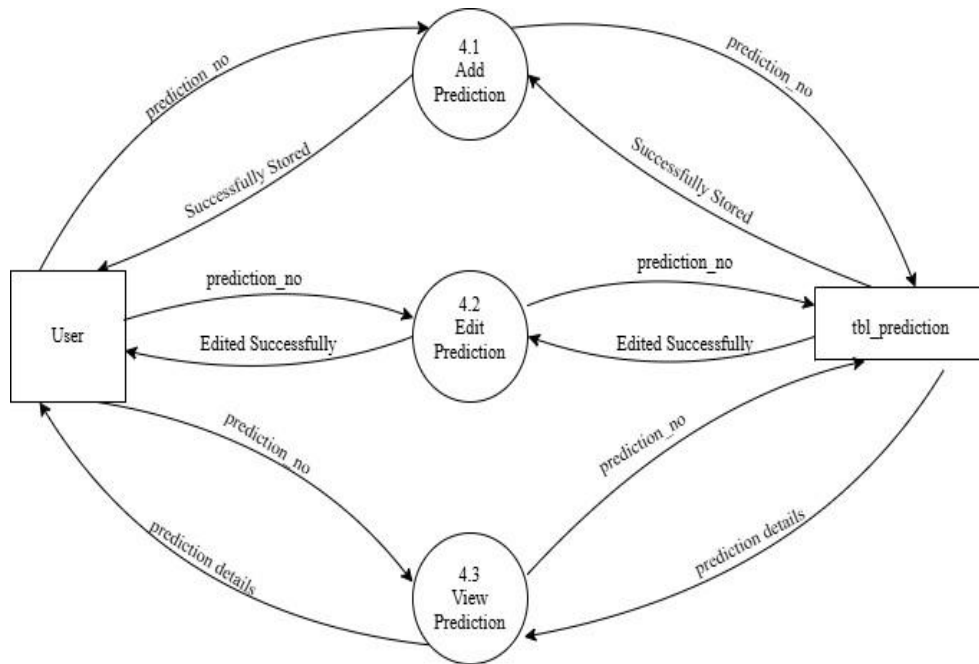
Level 2.1 DFD: Payment Management



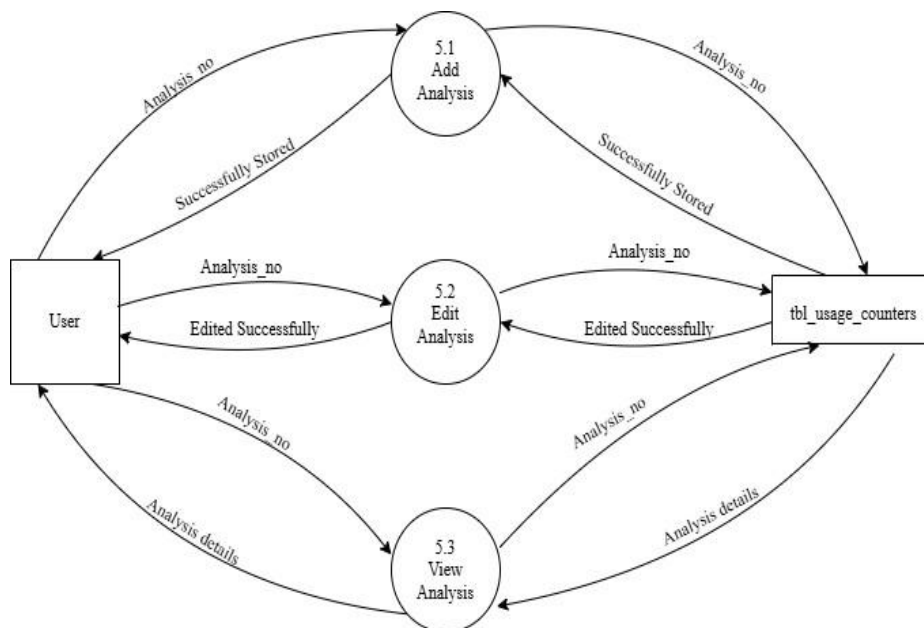
Level 2 DFD: Disease Information Management



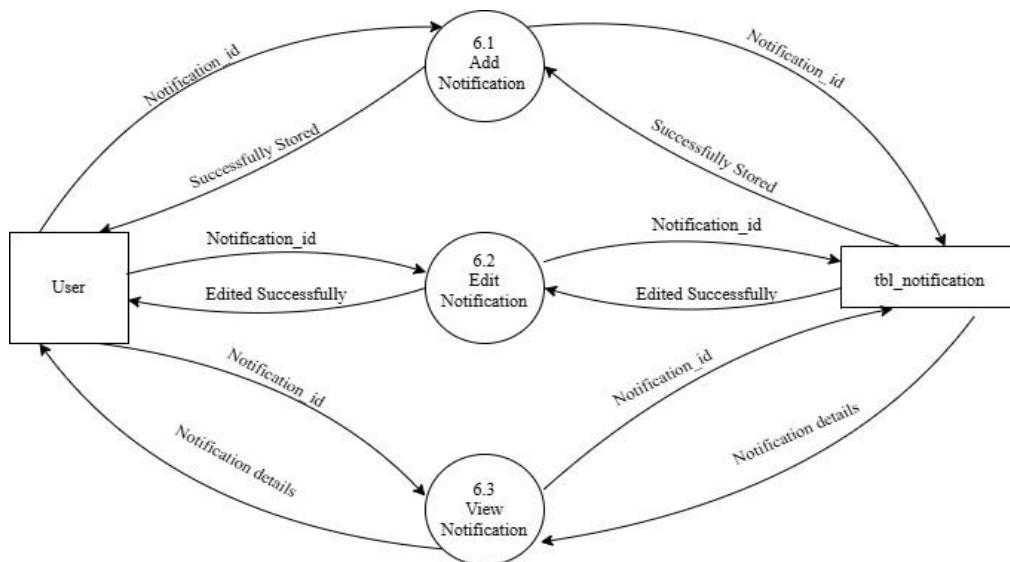
Level 2 DFD: Disease Prediction Management



Level 2 DFD: Disease Analysis Management

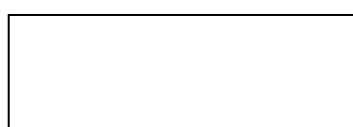


Level 2 DFD: Notification Management

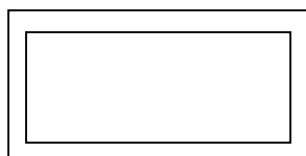


3.2 ENTITY RELATIONSHIP DIAGRAM

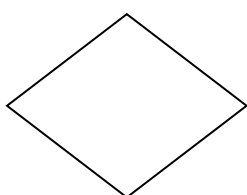
The ER model provides a conceptual way to perceive the real world, representing it as a collection of entities and the relationships between them. Central to this model is the Entity-Relationship diagram, a visual tool used to illustrate data components. ER modelling is extensively applied in the initial design phase of database applications.



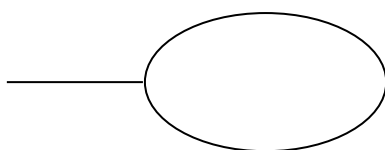
Entity Type



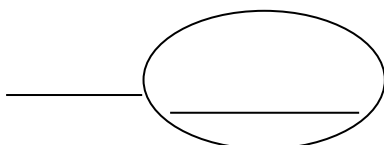
Weak Entity Type



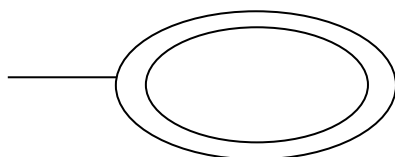
Relationship Type



Attribute

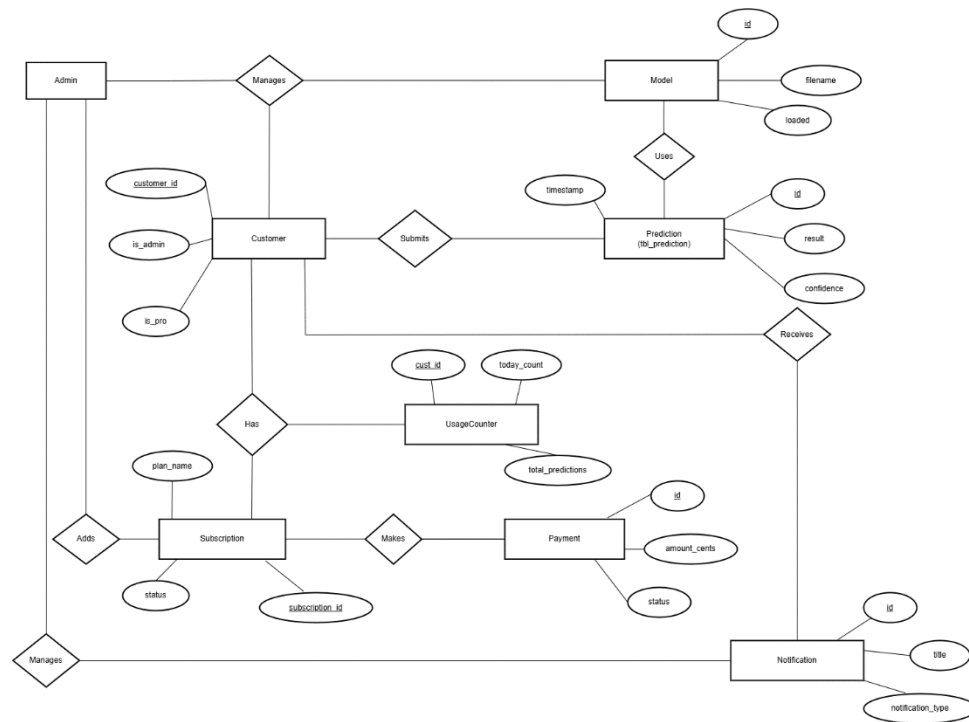


Key Attribute



Multivalued Attribute

Entity Relationship diagram for Plant Disease Detection and Analysis System



SYSTEM DESIGN

4.1 INPUT DESIGN

Input design is the process of converting a user-oriented description of the inputs to a computer-based system into a programmer-oriented specification. The quality of system input determines the quality of system output. Input specification describes the manner in which data enter the system for processing. Input design features can ensure the reliability of the system and produce result from accurate data or they can result in the production of errors. The input design also determines whether the user can interact efficiently with the system. Input design requires consideration of the needs of the data entry operator. Three data entry considerations are:

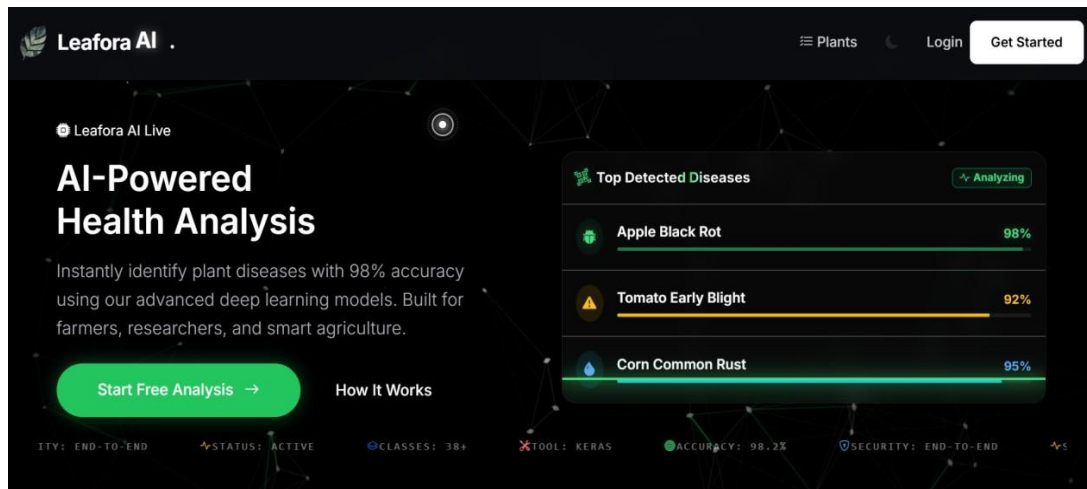
- The field length must be documented
- The sequence of fields must match the sequence of the fields on the source document.
- The data format must be identified to the data entry operator.

In our system almost, all inputs are being taken from the databases. To provide adequate inputs we have to select necessary values from the databases and arrange it to the appropriate controls.

Inaccurate input data are the most common cause of errors in data processing. Errors entered by data entry can be controlled by input design. Input design is the process of converting user-oriented inputs to a computer-based format. There are three major approaches for entering data into the computer. They are menus, formatted forms and prompts. A menu is a selection list that simplifies computer data access or entry. Instead of remembering what to enter, the user chooses from the list of option. A formatted form is a pre-printed form or a template that request the user to enter data inappropriate location. It is a fill-in- the-blank type form. The form is flashed on the screen as a unit. In prompt the system displays one enquiry at a time, asking the user for a response.

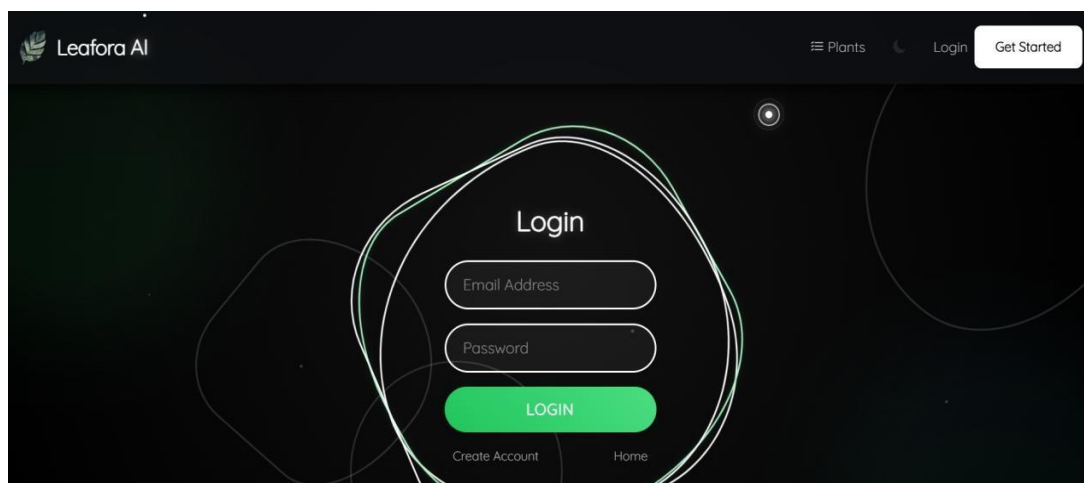
Home Page

Description: This is the home page for this website



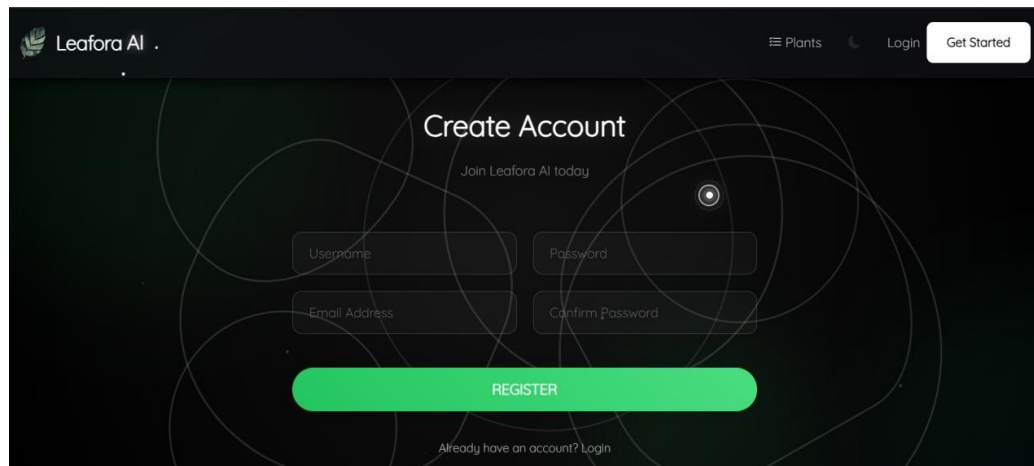
Login Page

Description: This is the login page for this website



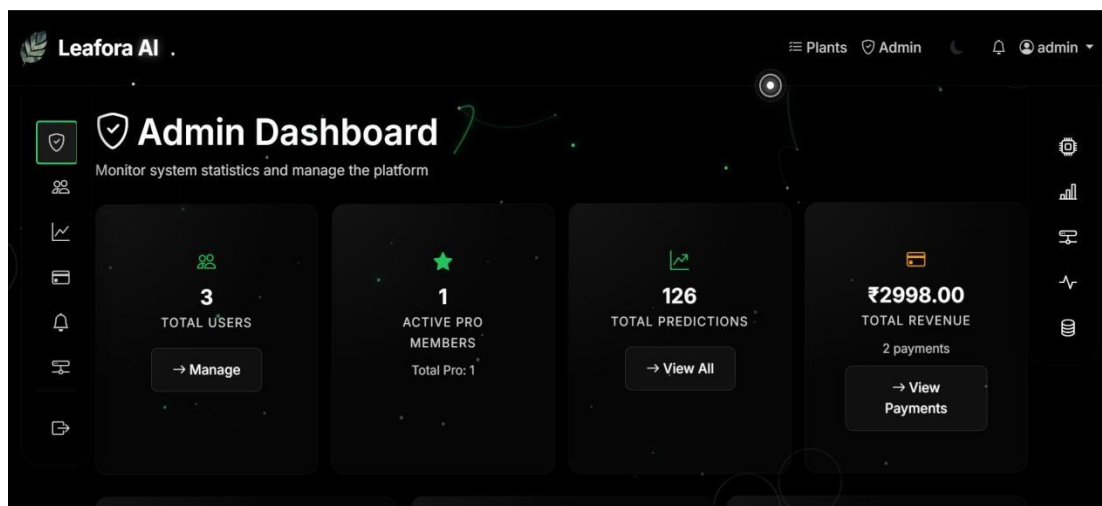
Customer Registration

Description: This is the customer registration page for this website



Admin Panel

Description: This is the page to manage programs page for this website



Customer Dashboard

Description: This is the page for the logged in customers for this website

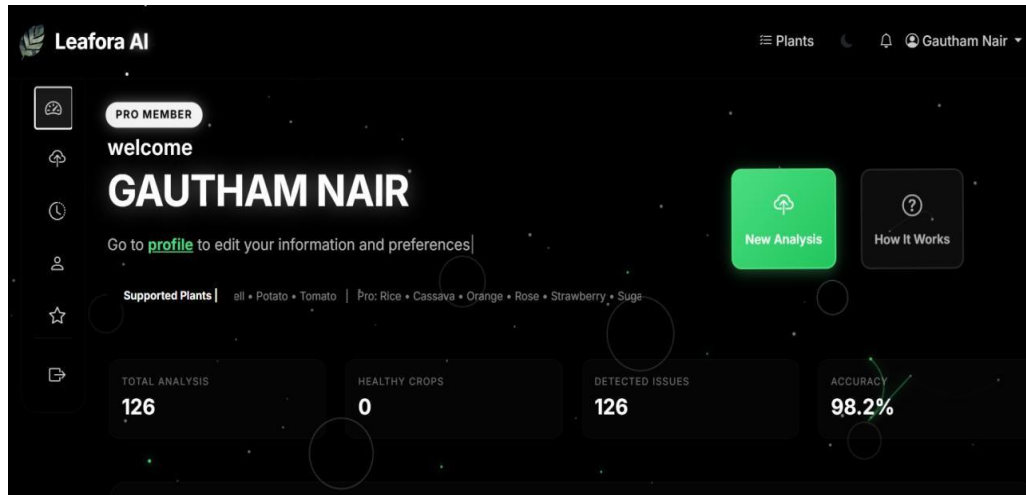
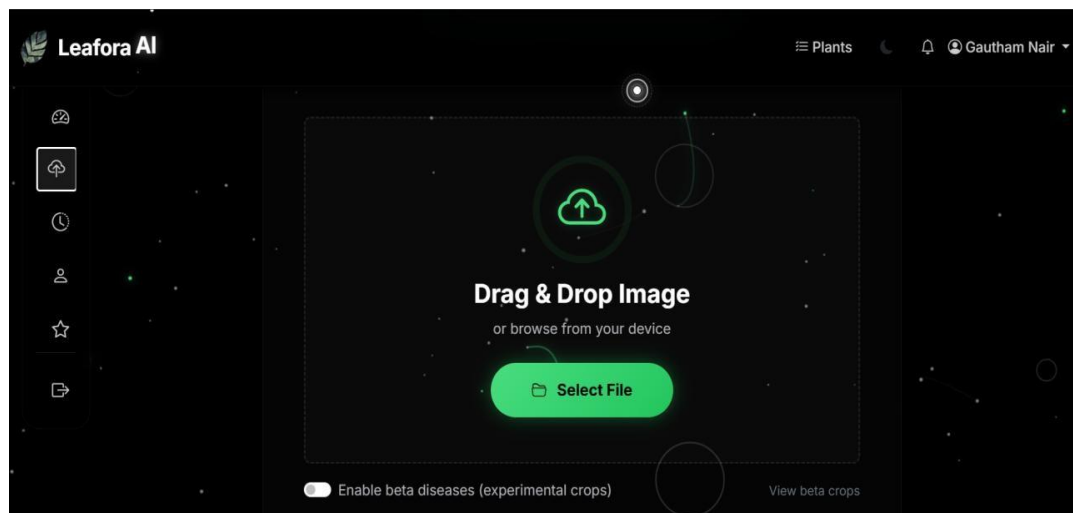


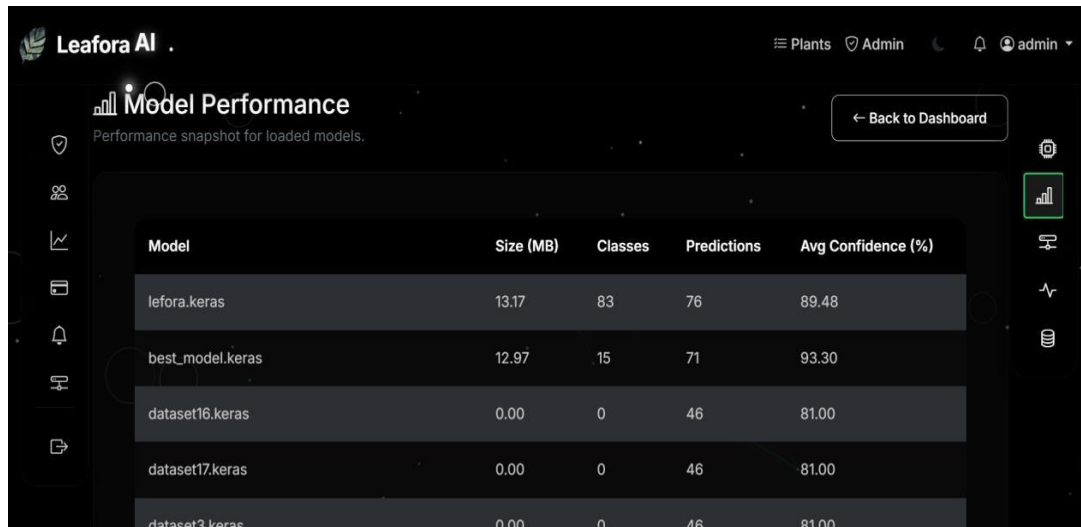
Image Input

Description: This is the page where customer upload the affected leaves for this website



Model Performance

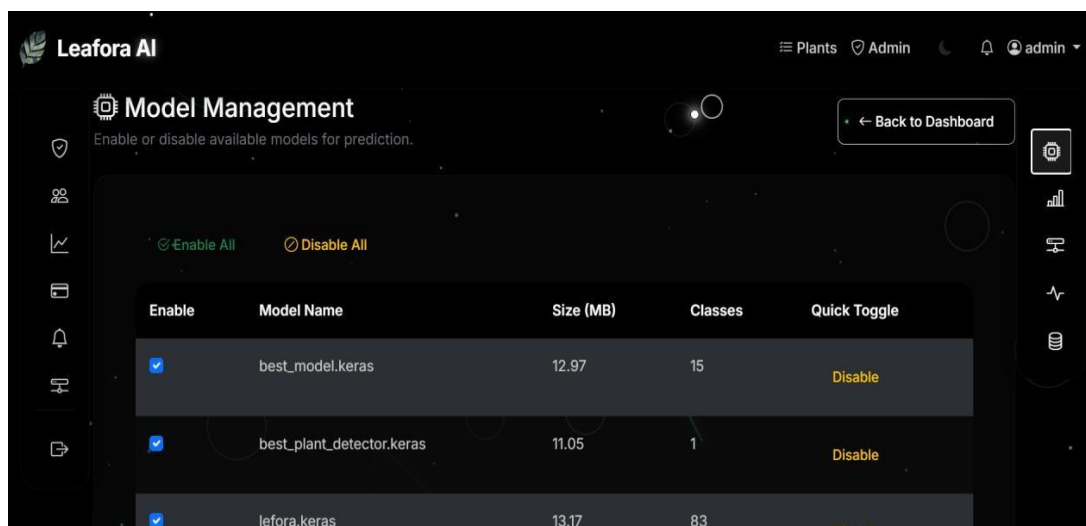
Description: This is the page to view the model details for this website



Model	Size (MB)	Classes	Predictions	Avg Confidence (%)
lefora.keras	13.17	83	76	89.48
best_model.keras	12.97	15	71	93.30
dataset16.keras	0.00	0	46	81.00
dataset17.keras	0.00	0	46	81.00
dataset3.keras	0.00	0	46	81.00

Manage Models

Description: This is the page to manage models for this website



Enable	Model Name	Size (MB)	Classes	Quick Toggle
☒	best_model.keras	12.97	15	Disable
☒	best_plant_detector.keras	11.05	1	Disable
☒	lefora.keras	13.17	83	Disable

4.2 OUTPUT DESIGN

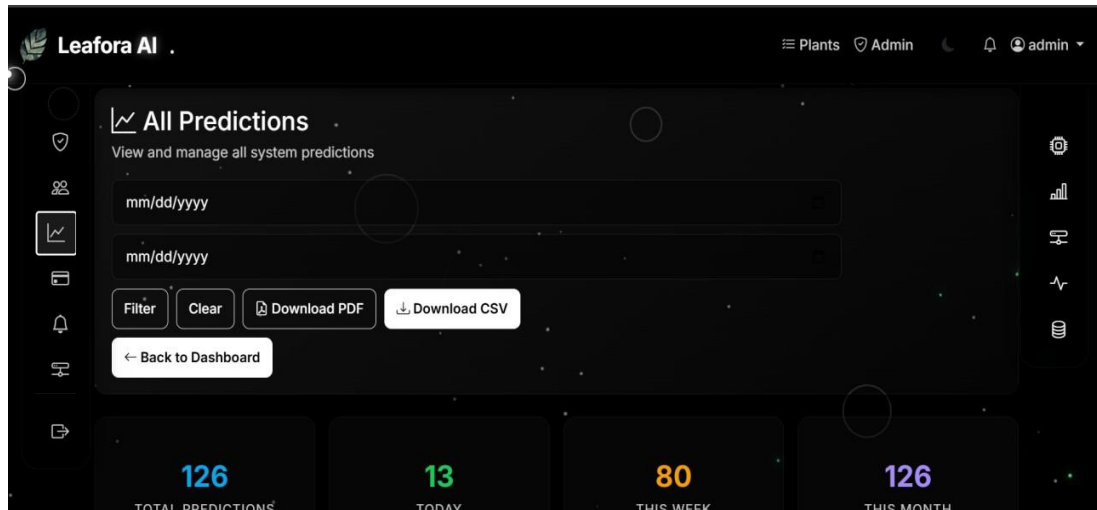
One of the important features of an information system for users is the output it produces. Output is the information delivered to users through the information system. Without quality output, the entire system appears to be unnecessary that users will avoid using it. Users generally merit the system solely by its output. In order to create the most useful output possible. One works closely with the user through an interactive process, until the result is considered to be satisfactory.

Output design has been an ongoing activity almost from the beginning of the project. In the study phase, outputs were identified and described general in the project directive. A tentative output medium was then selected and sketches made for each output. In the feasibility analysis, a “best” new system was selected; its description identified the input and output media. In the design phase the system has included an evaluation and selection of specific equipment for the system.

Outputs from computer systems are required primarily to communicate the results of processing to the user. They are also used to provide a permanent copy of these results for later consultation.

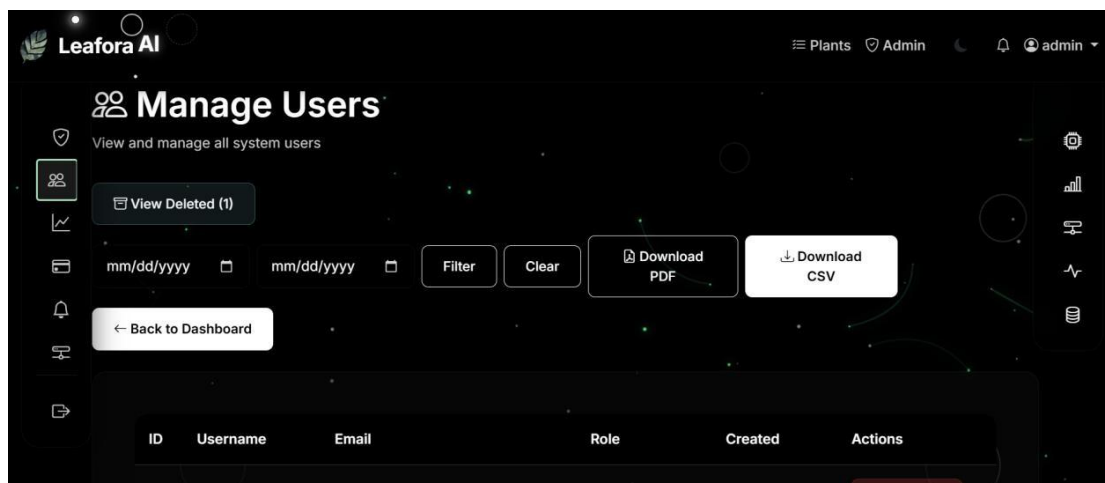
Predictions Report

Description: This is the page to manage predictions for this website



Customers Report

Description: This report shows the details of customers.



Subscription Report

Payment ID	Customer	Customer ID	Amount	Card	Status	Date	Expires	Actions
#2	Test User testing@gmail.com	4	\$1499.00 INR	**** **** ****	Completed	2026-02-16	N/A	Download

Activity Report

Timestamp	Type	Username	Email	Description
2026-02-17	Login	admin	admin@admin.com	admin logged in

4.3 DATABASE DESIGN

4.3.1 Normalization

Designing a database is a complex task and the normalization theory is a useful aid in this design process. The process of normalization is concerned with transformation of conceptual schema into computer representation form.

A bad database design may lead to certain undesirable situations such as,

- Repetition of information
- Inability to represent certain information
- Loss of information

To minimize these anomalies, normalization may be used. If the database is in a normalized form, the data can be restructured and can maintain it easily. This is important that the databases using that we are using may free from data redundancy and inconsistency. For this need we maintain the tables in a normalized manner.

First Normal Form

A relation is in first Normal Form (1NF), if and only if all its attributes are based on single domain. The objective of normalizing a table is in to remove its repeating groups and ensure that all entries of the resulting table have at most single value.

Second Normal Form

A table is said to be in second Normal Form (2NF), when it is in 1 NF and every attribute in the record is functionally dependent upon the whole key, and not just a part of the key.

Third Normal Form

A relation that is in First and Second Normal Form and in which no non-primary-key attribute is transitively dependent on the primary key, then it is in Third Normal Form (3NF).

TABLE DESIGN

Table No: 1

Table Name: tbl_customer

Field	Data Type	Constraint	Description
customer_id	INTEGER	PRIMARY KEY	Unique identifier for the customer
is_admin	BOOLEAN	NOT NULL	Checks if the user is Admin
is_pro	INTEGER	NOT NULL	Is Pro - 1 if Pro subscriber, 0 otherwise
pro_expires_at	DATETIME	NOT NULL	Date when Pro subscription expires
is_deleted	BOOLEAN	NOT NULL	if account is deleted/banned
created_at	DATETIME	NOT NULL	Account creation timestamp
last_login_at	DATETIME	NOT NULL	last login at
last_logout_at	DATETIME	NOT NULL	last logout at

subscription_purchased_at	DATETIME	NOT NULL	Date of latest subscription
email	VARCHAR(120)	NOT NULL	Email ID
password_hash	VARCHAR(255)	NOT NULL	Encrypted Password
subscription_id	INTEGER	FOREIGN KEY	ID of the active subscription

Table No: 2

Table Name: tbl_payment

Field	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Unique identifier
cust_id	INTEGER	FOREIGN KEY	Unique identifier for the customer
subscription_id	INTEGER	FOREIGN KEY	ID of the active subscription
card_name	VARCHAR(50)	NOT NULL	Card Holders Name
card_no	VARCHAR(20)	NOT NULL	Card Number
card_last4	VARCHAR(4)	NOT NULL	Last 4 digits
card_expiry	DATE	NOT NULL	Card Expiry

payment_date	DATETIME	NOT NULL	Date and time of payment
status	VARCHAR(20)	NOT NULL	Current Payment Status
receipt_path	VARCHAR(255)	NOT NULL	Path to the payment receipt file
payment_method	VARCHAR(50)	NOT NULL	Payment Method
transaction_id	VARCHAR(255)	NOT NULL	Transaction ID
expires_at	DATETIME	NOT NULL	When the subscription expires

Table No: 3

Table Name: tbl_subscriptions

Field	Data Type	Constraint	Description
subscription_id	INTEGER	PRIMARY KEY	ID of the active subscription
cust_id	INTEGER	FOREIGN KEY	Unique identifier for the customer
plan_name	VARCHAR(50)	NOT NULL	Subscription Name
plan_price	NUMERIC(10,2)	NOT NULL	Plan Price
plan_duration	INTEGER	NOT NULL	Plan Duration

daily_limit	INTEGER	NOT NULL	Daily Limit
plan_features	TEXT	NOT NULL	Plan Features
start_date	DATETIME	NOT NULL	Subscription start date
end_date	DATETIME	NOT NULL	Subscription end date
expires_at	DATETIME	NOT NULL	When the subscription expires
status	VARCHAR(20)	NOT NULL	Status
is_active	INTEGER	NOT NULL	Is Active - 1 if currently active
created_at	DATETIME	NOT NULL	Account creation timestamp
updated_at	DATETIME	NOT NULL	Last update timestamp

Table No: 4

Table Name: tbl_prediction

Field	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Unique identifier
cust_id	INTEGER	FOREIGN KEY	Unique identifier for

			the customer
filename	VARCHAR(255)	NOT NULL	Name of the model
result	VARCHAR(100)	NOT NULL	Disease prediction result
confidence	FLOAT	NOT NULL	Confidence score of the prediction
topk_results	TEXT	NOT NULL	top 3 predictions
models_used	TEXT	NOT NULL	List of models used for prediction
is_unknown	BOOLEAN	NOT NULL	1 if disease is unknown
timestamp	DATETIME	NOT NULL	Date and time of event

Table No: 5

Table Name: tbl_usage_counters

Field	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Unique identifier
cust_id	INTEGER	FOREIGN KEY	Unique identifier for the customer
filename	VARCHAR(255)	NOT NULL	Name of the model

result	VARCHAR(100)	NOT NULL	Disease prediction result
confidence	FLOAT	NOT NULL	Confidence score of the prediction
topk_results	TEXT	NOT NULL	top 3 predictions
models_used	TEXT	NOT NULL	List of models used for prediction
is_unknown	BOOLEAN	NOT NULL	1 if disease is unknown
timestamp	DATETIME	NOT NULL	Date and time of event

Table No: 6

Table Name: tbl_models

Field	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Unique Identifier
filename	VARCHAR(255)	NOT NULL	Name of the file
size_bytes	INTEGER	NOT NULL	Size of the model file
backend	VARCHAR(10)	NOT NULL	Framework used
classes	INTEGER	NOT NULL	Number of classes the model can detect

loaded	BOOLEAN	NOT NULL	1 if the model is loaded in the memory
created_at	DATETIME	NOT NULL	Model creation timre
Name	VARCHAR(255)	NOT NULL	

Table No: 7

Table Name: tbl_notifications

Field	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Unique identifier
cust_id	INTEGER	FOREIGN KEY	Unique identifier for the customer
title	VARCHAR(255)	NOT NULL	Notification title
message	TEXT	NOT NULL	Notification content
notification_type	VARCHAR(50)	NOT NULL	Category of notification
is_read	BOOLEAN	NOT NULL	1 if read, 0 if unread
is_system	BOOLEAN	NOT NULL	1 if system notification
created_at	DATETIME	NOT NULL	Notification Creation Timestap

TESTING

5.1 INTRODUCTION

Testing is the process of examining the software to compare the actual behaviour with that of the expected behaviour. The major goal of software testing is to demonstrate that faults are not present. In order to achieve this goal, the tester executes the program with the intent of finding errors. Though testing cannot show absence of errors but by not showing their presence it is considered that these are not present.

System testing is the first Stage of implementation, which is aimed at ensuring that the system works accurately and efficiently before live operations commences. Testing is vital to the success of the system. System testing makes a logical assumption that if all the parts of the system are correct and the goal will be successfully achieved. A series of testing are performed for the proposed system before the proposed system is ready for user acceptance testing.

Levels of Testing

5.1.1 Unit Testing

5.1.2 Integration Testing

5.1.3 Output Testing

5.1.4 Validation Testing

5.1.1 Unit Testing

In the Leafora AI project, unit testing was performed to verify each module independently before integration. The user registration module was tested for required-field checks, duplicate username/email handling, and correct user/customer record creation. The login module was verified using valid and invalid credentials, including banned-user restrictions. The image upload module was tested for file-type validation, corrupted/empty image rejection, dimension checks, and image normalization. The prediction module was checked for top-k disease classification, non-leaf detection, and confidence generation. The payment validation module was tested for card number, expiry date, CVV, and cardholder-name rules. This process helped identify and resolve module-level defects early.

5.1.2 Integration Testing

Integration testing in Leafora AI ensured smooth interaction among all modules. Newly registered users were tested for successful login and dashboard access using the same credentials. Uploaded images were validated through the complete pipeline: upload, preprocessing, disease prediction, database storage, and history display. The subscription workflow was tested end-to-end to verify payment processing, Pro activation, notification creation, and receipt generation. Admin-side functions were tested to confirm that updates in users, predictions, and payments appeared correctly across dashboards and reports. This validated consistent data flow across authentication, prediction, subscription, and administration modules.

5.1.3 Output Testing

Output testing focused on correctness and presentation of generated results. Prediction output pages were verified for accurate disease label, confidence score,

top-k alternatives, and recommendation details. Non-plant images were tested to ensure they produced a “not a plant image” response instead of a false disease diagnosis. User history and dashboard outputs were checked for correctness of records and statistics. Downloadable outputs, including prediction PDFs, payment receipts, and admin CSV/PDF reports, were reviewed for completeness, formatting, and readability. This ensured reliable and professional system outputs.

5.1.4 Validation Testing

Validation testing confirmed that Leafora AI met its functional and access-control requirements. It verified that only authenticated users could upload images and run predictions. Ownership validation ensured users could access only their own results and receipts, while admin routes remained restricted to administrators. Subscription validation confirmed daily limits for Basic users and correct feature access for Pro users. Payment callback/webhook validation ensured Pro status was granted only after successful payment confirmation. This verified that the system behavior aligned with its intended purpose and security requirements.

5.2 TEST CASES

A test plan documents the strategy that will be used to verify and ensure that a product or system meets its design specification and other requirements. A test plan is usually prepared by or with significant input from Test Engineers.

5.2.1 Unit Testing

Form	Procedure	Expected Result	Actual Result	Status
Login Form	Log in with valid customer credentials	Redirected to user dashboard	Login successful, user dashboard loaded	Pass
Login Form	Log in with valid admin credentials	Redirected to admin dashboard	Login successful, admin dashboard loaded	Pass
User Registration Form	Submit completed registration details	New user, customer, and usage counter records created	Account created successfully	Pass
Upload Form	Upload valid leaf image	Image accepted and processed (.jpg/.png)	Upload and preprocessing successful	Pass
Upload Form	Upload invalid file type (.pdf/.txt)	Upload rejected with validation message	Validation message shown	Pass
Prediction Module	Run prediction on valid	Top disease result with confidence	Prediction generated	Pass

	uploaded image	returned	correctly	
Card Payment Validation	Submit valid card details	Payment accepted and Pro activated	Pro plan activated	Pass
PDF Report Download	Download prediction PDF from history/result	PDF generated and downloaded	PDF downloaded successfully	Pass
5.2.2 Integration Testing				
Form / Flow	Expected Result	Actual Result	Status	
Registration to Login	Newly registered user can log in with same credentials	Login successful with registered credentials	Pass	
Login to Dashboard	Role-based dashboard loads after login	Correct dashboard loaded by role	Pass	
Upload Prediction History	Uploaded image is predicted and saved in user history	Prediction appears in history	Pass	
Subscription Payment to Pro Access	Successful payment updates plan and enables Pro features	Pro status updated and features enabled	Pass	
Admin Notification to User Panel	Admin-created notification appears in user notifications	Notification shown with unread count update	Pass	
Admin User Block to Login	Deleted/blocked user cannot access login session	Login denied for blocked user	Pass	
Prediction	Users can only access their	Unauthorized access	Pass	

Ownership to Result/PDF	own results and reports	blocked	
5.2.3 Validation Testing Table			
Form	Expected Result	Actual Result	Status
Authentication Validation	Only logged-in users can access upload, history, and prediction routes	Anonymous access redirected to login	Pass
Authorization Validation	Only admins can access admin routes	Non-admin users blocked from admin pages	Pass
File Validation	Only allowed image formats are accepted; invalid/corrupt files rejected	Validation and rejection working correctly	Pass
Usage Limit Validation	Basic users are restricted by daily prediction limit	Limit enforced and upgrade prompt shown	Pass
Pro Feature Validation	Pro-only options are available only for active Pro users	Feature gating works correctly	Pass
Payment Validation	Pro activation occurs only after successful payment confirmation	Status updated only on verified payment	Pass
Data Access Validation	Users can view/download only their own receipts and prediction reports	Cross-user access prevented	Pass
Non-Leaf Validation	Non-plant images should not produce disease diagnosis	"Not a plant image" flow triggered correctly	Pass

IMPLEMENTATION

6.1INTRODUCTION

Implementation is the phase where the planned design of the Leafora AI Plant Disease Detection System is transformed into a fully functional, working software application. This stage ensures that the system performs reliably and satisfies user expectations in a real-world environment.

For Leafora AI, implementation involved building and configuring a web-based platform using Python Flask, SQLite/MySQL-compatible database workflows, and integrated AI model pipelines for crop disease identification. The implemented modules include user registration and login, image upload and validation, disease prediction, prediction history, PDF report generation, subscription and payment handling, notifications, and admin monitoring dashboards. These modules were tested to ensure smooth end-to-end operation, where users could upload crop images and receive diagnosis results, while administrators could manage users, view analytics, and monitor transactions and logs.

In addition, role-based access control was implemented so that each type of user (Admin and Member/User) can access only the features relevant to their responsibilities. Supporting guidance through UI prompts and structured workflows was included to improve usability for both users and administrators. After successful testing and verification, the system was prepared for deployment, enabling faster, more accurate, and organized disease detection and platform management.

6.2 INSTALLATION PROCEDURE

The installation of the Leafora AI Plant Disease Detection System involves preparing the Python server environment, configuring the database, loading model files, and deploying the web application for user access. Steps followed during installation:

Environment Setup: Installed the required runtime stack: Python, pip, and supporting libraries for Flask, SQLAlchemy, image processing, and AI inference.

Project Deployment: Placed the project files in the application directory (for example, e:\main project) with folders such as modules, templates, static, migrations, and model directories.

Dependency Installation: Created/activated a virtual environment and installed all required packages from requirements.txt to ensure consistent execution.

Configuration Setup: Configured application settings (for example, SECRET_KEY, database URI, API keys) through environment variables and project config files.

Database Configuration: Initialized and migrated the database schema using Flask Migrate/Alembic, creating required tables such as

tbl_login, tbl_customer, tbl_predictions, tbl_payments, tbl_usage_counters, tbl_notifications, and tbl_models.

Model Setup: Added trained model files (.keras/.h5) to the expected model directories (m_models, detection, or configured paths) so prediction services could load successfully.

Static/Upload Path Preparation: Ensured required runtime directories (such as static/uploads and static/pdfs) were available for image uploads and report generation.

Role Initialization: Configured role-based access for Admin and Member/User operations. The initial admin account was verified and secured for operational

use.

Application Launch: Started the system using the Flask entry point (for example, python app.py or run_app.bat) and accessed it through a browser.

Initial Verification Testing: Performed startup test runs for registration, login, image upload, prediction, history, PDF download, and admin monitoring modules to confirm system stability and correctness. After successful completion, the system was made accessible through web browsers on desktop/laptop systems within local or institutional network environments.

6.3 IMPLEMENTATION PLAN

The implementation plan defines the transition from development to live deployment of the Leafora AI Plant Disease Detection System.

Phase 1 – Preparation: Set up the runtime environment (Python, virtual environment, required libraries), configure application settings, and prepare database/migration tooling.

Phase 2 – Module Deployment: Deploy and verify each core module in sequence: Authentication, Image Upload & Validation, AI Prediction Engine, History & Result View, PDF Report Generation, Subscription & Payment, Notifications, and Admin Monitoring modules.

Phase 3 – User Setup: Create and verify user roles (Admin and Member/User), initialize admin access, and confirm route-level access restrictions and ownership checks for sensitive data (results, receipts, admin pages).

Phase 4 – Operational Readiness and Training: Provide usage guidance for administrators and operational users, including account handling, prediction review, report download, payment monitoring, and notification workflows.

Phase 5 – Final Testing and Go-Live: Execute real-world validation scenarios such as user registration/login, image upload and disease detection, history and PDF export, subscription activation, and admin reporting. After successful verification, deploy the system for live institutional use.

Through this structured implementation process, the Leafora AI system was deployed as a reliable and user-friendly platform for crop disease detection and administrative monitoring.

CONCLUSION

7 FUTURE ENHANCEMENTS

While the Leafora AI Plant Disease Detection System has been successfully implemented with its core features, several enhancements can further improve accuracy, scalability, and user experience.

A major next step is to develop a dedicated mobile application for Android and iOS, enabling farmers and field users to upload crop images, view diagnosis results, and receive instant push alerts directly from mobile devices. The prediction pipeline can be expanded with stronger multi-model orchestration, region-specific crop support, and offline-capable lightweight inference models for low-connectivity environments. For administrators, advanced analytics dashboards can be enhanced to provide trend insights such as crop-wise disease distribution, seasonal outbreak patterns, model confidence behavior, and subscription/revenue statistics for better decision-making.

To strengthen reliability and security, future versions can include multi-factor authentication, stronger audit controls, encrypted storage policies, and stricter API key governance. A key infrastructure enhancement is cloud migration (AWS, Azure, or Google Cloud) for scalable deployment, centralized monitoring, high availability, automated backups, and disaster recovery. The system can also evolve into a multi-tenant or multi-organization platform, allowing centralized administration for multiple institutions, regions, or agricultural centers.

Looking ahead, Leafora AI can integrate advanced AI capabilities such as personalized treatment recommendations, crop-stage-aware guidance, weather-linked risk alerts, and early outbreak forecasting using historical and environmental data. These enhancements would transform the platform into a more intelligent, scalable, and decision-support-oriented ecosystem for both end users and administrators.

BIBLIOGRAPHY

Text Books

The following textbooks were referred to while developing this project:

Silberschatz, Abraham, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. 7th Edition, McGraw-Hill Education, 2020.

Pressman, Roger S., and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. 8th Edition, McGraw-Hill, 2019.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd

Edition, O'Reilly Media, 2022.

Websites

The following websites were referred to for development and report preparation:

<https://stackoverflow.com>

For troubleshooting code-related issues.

<https://docs.python.org>

For official Python documentation.

<https://flask.palletsprojects.com>

For Flask framework documentation.

<https://www.sqlalchemy.org>

For SQLAlchemy ORM and database integration references.

<https://alembic.sqlalchemy.org> For database migration guidance.

<https://www.tensorflow.org>

For TensorFlow implementation references.

<https://keras.io>

For Keras model-building documentation.

APPENDICES

APPENDIX A

Sample source code/pseudo code Login Page Code

```
@app.route('/login',
methods=['GET', 'POST'])
def login():

    if 'user_id' in session:

        return redirect(url_for('dashboard'))


    if request.method == 'POST':

        email    =    request.form.get('email',
        "").strip().lower()    password    =
        request.form.get('password', "")

        if not email or not password:

            flash('Please enter both email and
            password.', 'danger')    return
            render_template('auth/login.html')


        user    =
        User.query.filter_by(email=
        email).first() if not user:

            flash('Invalid email or
            password.', 'danger')
            return
```

```
render_template('auth/login.html')

if hasattr(user, 'is_deleted') and user.is_deleted:

    flash('User has been banned from the session. Please contact administrator.', 'danger')

    return render_template('auth/login.html')

if
    check_password_hash(user.password_hash, password):
        session['user_id'] = user.id

        session['username'] = user.username
        session['is_admin'] = user.is_admin
        return redirect(url_for('dashboard'))

    flash('Invalid email or password.', 'danger')
    return render_template('auth/login.html')

return render_template('auth/login.html')
```

Login Page Template

```
<form method="POST" action="{{ url_for('login') }}">  
  
  <input type="email" name="email" placeholder="Email" required>  
  
  <input type="password" name="password" placeholder="Password"  
  required>  
  
  <button type="submit">Sign In</button>  
  
</form>
```

Image Upload & Prediction Module Code

```
@app.route('/upload', methods=['GET', 'POST']) @login_required  
@member_only def upload():  
    user = db.session.get(User, session['user_id'])  
  
    if request.method == 'POST':  
  
        can_predict, msg = check_usage_limit(user) if not can_predict:  
  
        flash(msg, 'warning')  
  
        return redirect(url_for('plans'))  
  
  
    file = request.files.get('file')  
  
    if not file or file.filename == "": flash('No file selected.', 'danger') return  
    redirect(url_for('upload'))  
  
  
    if not allowed_file(file.filename):  
  
        flash('Invalid file type. Please upload an image.', 'danger') return  
        redirect(url_for('upload'))  
  
  
    filename = build_unique_upload_filename(file.filename)
```

```
filepath      =      os.path.join(app.config['UPLOAD_FOLDER'],      filename)
file.save(filepath)

ok, err = validate_image_file(filepath) if not ok:

flash(err, 'danger')

return redirect(url_for('upload'))


result = predict.predict_topk(filepath, k=predict.TOP_K)

top1 = result['topk'][0] prediction = Prediction(

cust_id=user.customer.customer_id,      filename=filename,      result=top1['label'],
confidence=top1['confidence'],

topk_results=json.dumps(result['topk']),      is_unknown=result.get('unknown',
False))

db.session.add(prediction) db.session.commit()

return      redirect(url_for('result',      prediction_id=prediction.id))      return
render_template('user/upload.html')
```

APPENDIX B

Acronyms

AI - Artificial Intelligence

API - Application Programming Interface CNN - Convolutional Neural Network

DBMS - Database Management System DFD - Data Flow Diagram

ER - Entity Relationship ML - Machine Learning

SQL - Structured Query Language UI - User Interface

UX - User Experience